UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Sistemas.

Estructura de datos

Prof. Gilberto A. Salinas

Semana 02
Apuntadores

Ciclo 2026-1

Agenda

1. Introducción
2. Memoria dinamica
   - Características
3. Referencias

1. Introducción: Direcciones de memoria

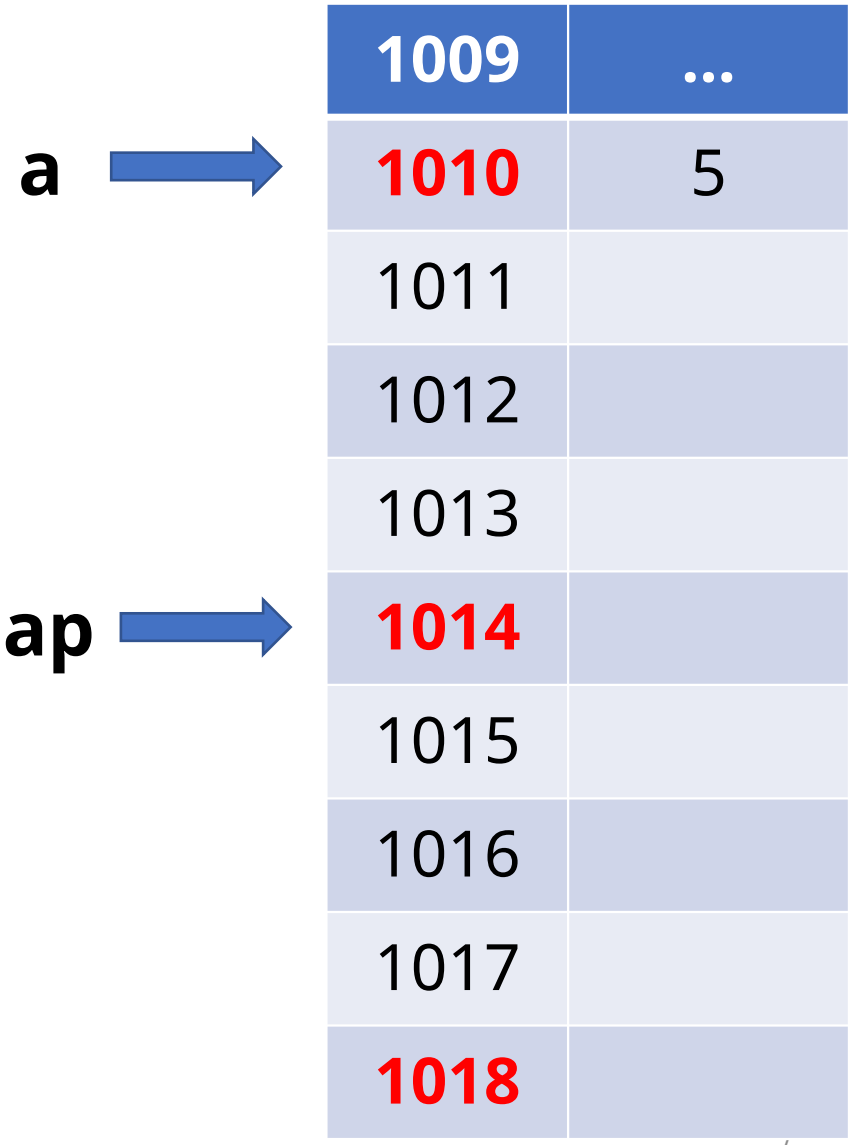
1. **Direcciones de memoria**. Todo dato se almacena en memoria utilizando varios bytes a partir de una dirección de memoria.
   - **int a = 5;**
2. **El dato (a): ROM EROM, RAM**
   - Se accede a partir de la dirección base (**1010**).
   - Dirección de la primera celda de memoria utilizada por el dato.
   - El tipo de dato (int) indica cuantos bytes (4) requiere el dato

     00000000 00000000 00000000 00000101
   - La codificación de los datos y de las direcciones pueden ser diferentes.

1009	...
1010	5
1011	
1012	
1013	
1014	...
1015	
1016	
1017	
1018	

1. Introducción: Variables apuntador

1. **Los apuntadores contiene direcciones de memoria.** Un apuntador sirve para acceder a través de otro dato.
   - El valor de apuntador es la dirección de memoria base del otro dato.
   - **int *ap, a=5;**
   - **//ap = &a; // *ap = 99**
2. **pIndirección.** Acceso indirecto a un dato (ap), (ap) ahora apunta a la dirección a (**1010**)
   - ¿De que tipo es el dato apuntado?.
   - El asterisco (*) indica que es un apuntador a datos de ese tipo: int *ap; float *fp;



1. Introducción: Apuntadores

1. **Un Apuntador**. Al declararse un variable de tipo apuntador contiene direcciones no validas. Requiere:
   - Ap = &a; //Ahora tiene una dirección valida.
   - Puede apuntar a NULL: ap=NULL;
2. **Utilidad de los apuntadores**
   - Paso de parámetros por referencia
   - Manejar datos dinámicos. Datos que se crean y destruyen en tiempo de ejecucion.
   - Implementación de arreglos. Int *a, **b, ***c;
   - C= new int[]

1. Introducción: Apuntadores multinivel

1. **Los apuntadores mutinivel en C/C++.** Son variables que almacenan la direccion de memoria, se definen por el nmero de asteriscos (*) lo cual indica el grado de indireccion.
   - **int *p;** Amacena la dir de una var simple
   - **int **p;** Amacena la dir de un apunt simple
   - **int ***p;** Amacena la dir de un apunt doble
2. Utilidad:
   - Matrices dinamicas.
   - Modificacion de apuntadores en subprogramas.
   - Estructura de datos complejas.
   - Arreglo de cadenas de texto.

Ejemplo:

- Int valor = 10;.
- Int *p = & valor;
- Int **p = & p;
- Int ***p = & pp;

Para profundizar vease el lab de apuntdores y consultar la documentcion cppreference.com

1. Introducción: Obtener lo que hay en la direccion

1. **Operador monario y prefijo**. (*) accede a lo que hay en la dirección apuntada.
   - Ap = &a; //Ahora tiene una dirección valida.
   - Muestra el valor de la variable apuntada: printf(“%d”, *ap); cout << *ap;
   - Acceso indirecto a la variable apunta a.

2. **Utilidad de los apuntadores**
   - Paso de parámetros por referencia
   - Manejar datos dinámicos. Datos que se crean y destruyen en tiempo de ejecucio.
   - Implementación de arreglos. Int *a, **b, ***c;

1. Introducción

1. **Apuntador**. Variable que almacena una dirección de memoria
   - Ap = &a;
2. **Declaracion de apuntadores**
   - TD *nombreVarApuntador
   - Int a, *ap;
   - NULL: valor nulo de un apuntador.
3. **Operador de dirección de memoria (&)**. Sobre objetos de memoria.
   - Direccion de memoria de a: &a
4. **Operador de indireccion (*)**. Sobre operadores que representan direcciones de memoria.
   - Contenido “a”: *(&a)

1. Introducción

1. **Reserva de espacio estático**. En la declaración de variables se hace una reserva e satica de espacios en la memoria para depositar valores en el , de lo contrario no podríamos utilizarlo.
2. **Reserva de memoria dinámica**. Se realiza en tiempo de ejecución.

2. Memoria dinámica

1. **Funciones de gestión memoria dinámica en C**
   - Reserva memoria: **void* malloc(size_t);**
     - Declaración: **NODO *p; *char car; int/int , (float)int/int**
     - Implementación: **p = (NODO*)malloc(sizeof(NODO));**
     - Implementación: **car = (char*)malloc(sizeof(char));**
   - Libera memoria: **free(void*);**
     - Implementación: **free(car); free(p);**
   - Redimensiona el espacio de memoria: **void* realloc(void*, size_t);**
     - Implementación: **car=realloc(car, sizeof(char)*n);** Donde n es el numero posiciones a ampliar.

2. Memoria dinámica

1. Funciones de gestión de memoria dinámica en C++

- Declaración: **NODO *p; *char car**
- Reserva memoria: **p = new(NODO);**
- Reserva memoria: **car = new(char);**
- Libera memoria: **delete p; delete car;** o también: **delete [] p; delete [] car;**

2. Memoria dinámica

1. Funciones de gestión de memoria dinámica en Java.

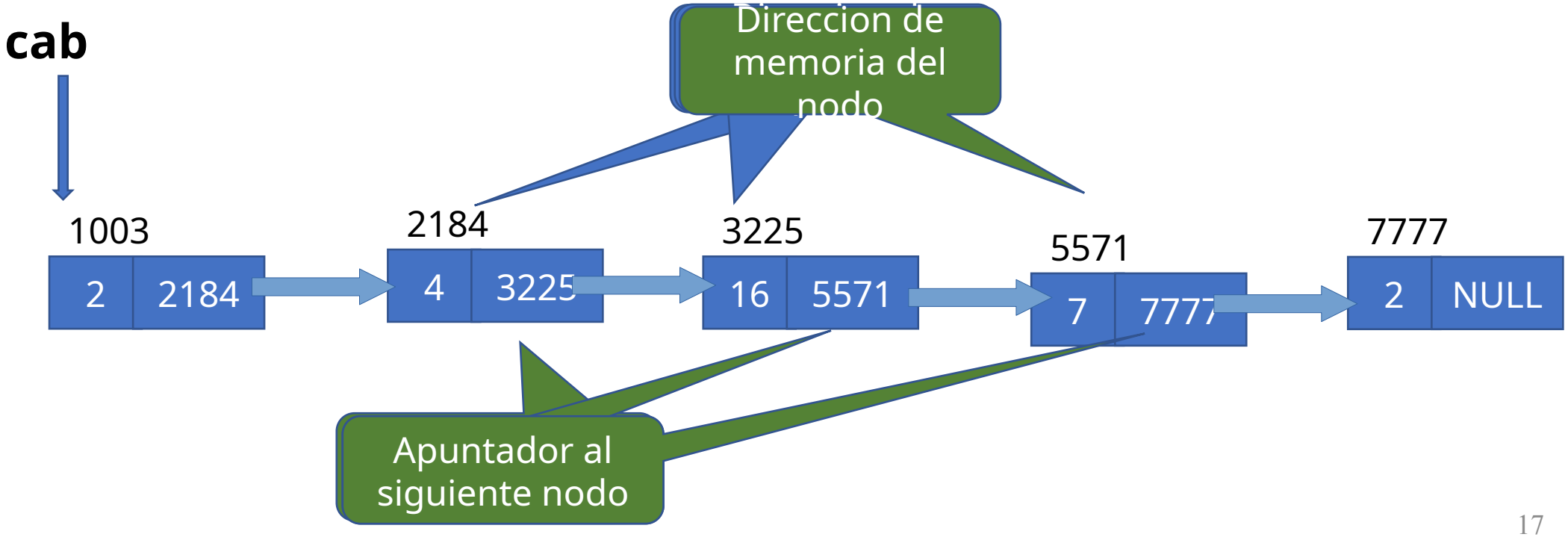
- **Memoria estática**. La asignación de memoria para elementos fijos es controlado por el compilador.
  - Define la cantidad necesaria para un programa en tiempo de compilación, el tamaño no puede cambiar.
  - Estos elementos se almacena en direcciones fijas.
  - Estas se almacenan e un espacio de memoria llamado **STACK**.
- **Memoria dinámica.** La asignación y la posible recuperación de memoria durante la ejecución de un programa y bajo su control se denomina asignación de memoria dinámica.
  - Define el tamaño espacio de memoria necesario en tiempo de ejecución.
  - El tamaño de los elementos pueden cambiar durante la ejecución del programa.
  - Almacena todos los elementos definidos con la palabra ***NEW*** en un programa
  - Estas se almacenan e un espacio de memoria llamado **HEAP.** Se libera a través de **garbage collector.**

1. Listas Enlazadas

1. **Introducción** . Son estructuras de datos fundamentales que pueden ser usadas para implementar otras estructuras de datos.
   - Fueron desarrolladas por Cliff Shaw y Herbert Simon en 1956
   - Los lenguajes imperativos u orientados a objetos como C, C++, Java, Python, … disponen de referencias para crear listas enlazadas.

2. Definicion

1. **Definición** . Es una estructura de datos lineal dinámica de un conjuntos de datos homogéneos ligados entre si por una referencia al siguiente elemento, conformada de nodos donde cada nodo contiene la información que puede almacenar y un apuntador al siguiente nodo.



2. Definicion

1. **Características**.
   - Son estructuras dinámicas que se crean y se eliminan en tiempo de ejecución
   - Pueden expandirse y contraerse fácilmente (inserciones y eliminaciones)
   - Se almacena en posiciones de memoria no contiguas.
   - El acceso a los elementos es secuencial.

2. Ventajas y Desventajas

1. **Ventajas**.
   - Son dinámicas, no necesita saber el numero de elementos a almacenar.
   - Son estructuras flexibles
   - Pueden expandirse y contraerse fácilmente
2. **Desventajas**.
   - Por su acceso secuencial no son buenas para búsquedas en grandes cantidades de datos.

3. Estructuras de datos estáticas.

1. **Las estructuras de datos estáticas arreglo**. Son estructuras de elementos *ordenados*, *contiguos, estática, homogénea* y de *acceso* con el nombre y el índice del arreglo.
   - **Ordenados.** Podemos mencionar al primer, segundo o n-esimo elemento.
   - **Continuo**. Los elementos se almacenan de forma contigua en la memoria RAM.
   - **Estática**. Tienen un numero determinado de elementos y debe definirse en el momento de la declaración o definicio. Ej., v[100].
   - **Homogénea**. Todos los elementos son del mismo tipo.
   - **Acceso**. El accso a los elementos es por medio del nombre y un índice del vector. Ej., v[5] = 59; printf(“%5d”, v[5]);

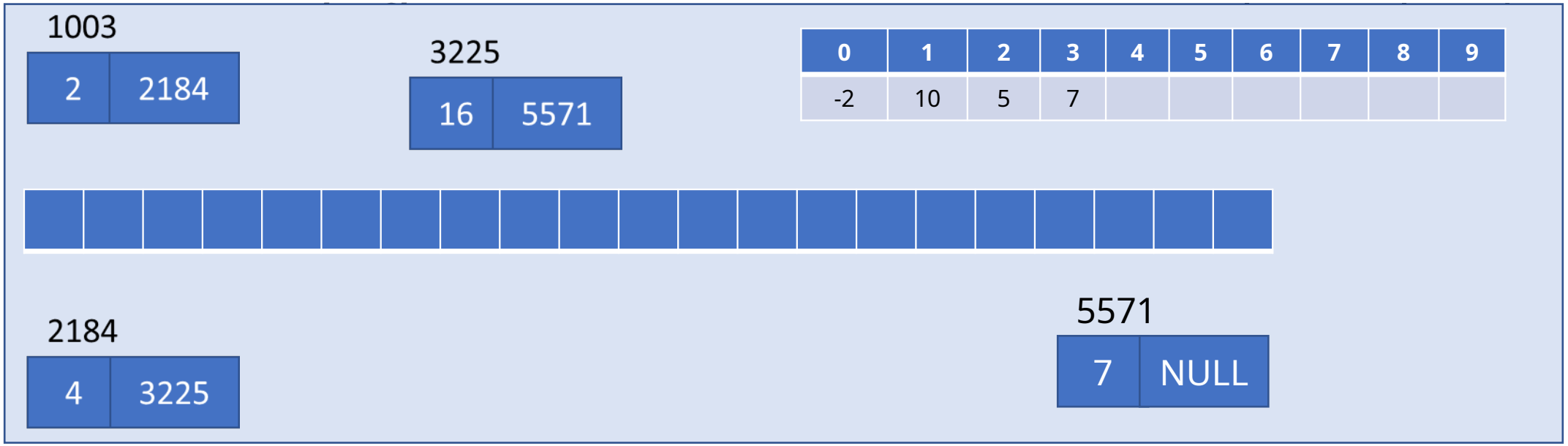
3. Estructuras estáticas.

1. **Las estructuras estáticas arreglo**. Características:
   - *El tiempo de acceso al primer o ultimo elemento es constante.*
   - *Es mas eficiente en búsquedas que para la inserción o eliminación por los desplazamientos de elementos que tiene que hacer en el vector.*
   - *El almacenamiento lógico y físico es el mismo.*

3. Estructuras estáticas.

1. **Representación lógica y física del arreglo**. Ej., int v[10], x[100]:
   - on: vector v, dv=4

0	1	2	3	4	5	6	7	8	9
-2	10	5	7						

   - 

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Villalobos J. (1996) Diseño y Manejo de Estructuras de Datos en C. Santa Fe de Bogota, Colombia: McGraw-Hill.
- Jensen T. (2000) Tutoría sobre apuntadores y arreglos en C. https://www.cimat.mx/~alram/cpa/pointersC.pdf